

ЛАБОРАТОРНАЯ РАБОТА № 6

«Введение в многопоточность»

Студент(ка): Зырянова Татьяна Евгеньевна

Группа: РИМ – 150950

Предмет: Программирование на Java

Репозиторий ДЗ: https://github.com/ZyrT12/java-core_2026-2027

ЦЕЛЬ РАБОТЫ

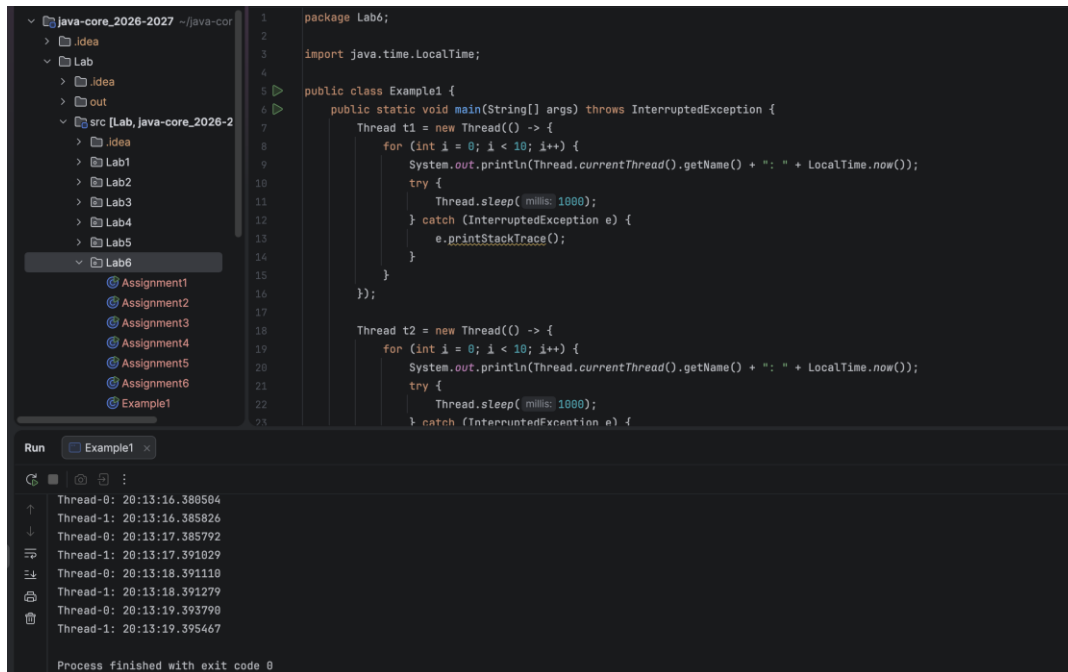
Получить представление о механизме многопоточности в языке программирования Java.

ОПИСАНИЕ ЗАДАЧИ

В данной лабораторной работе необходимо изучить механизм многопоточности в языке Java. В ходе выполнения работы требуется реализовать примеры создания и запуска потоков, использовать методы `sleep()` и `join()`, организовать синхронизацию потоков.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

1) Напишите программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.



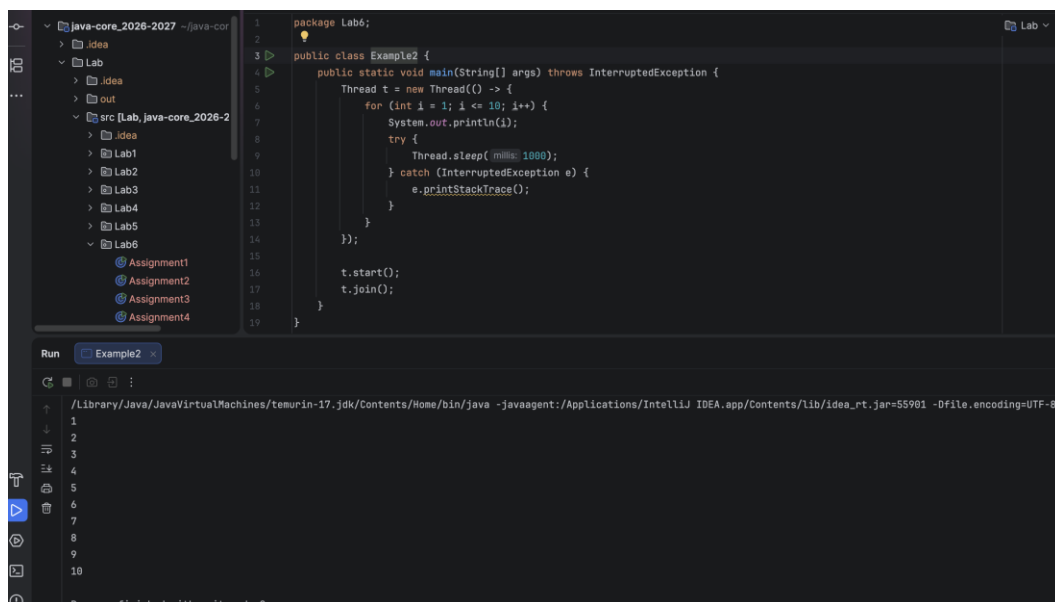
```
1 package Lab6;
2
3 import java.time.LocalDateTime;
4
5 public class Example1 {
6     public static void main(String[] args) throws InterruptedException {
7         Thread t1 = new Thread() -> {
8             for (int i = 0; i < 10; i++) {
9                 System.out.println(Thread.currentThread().getName() + ": " + LocalDateTime.now());
10                try {
11                    Thread.sleep(1000);
12                } catch (InterruptedException e) {
13                    e.printStackTrace();
14                }
15            }
16        });
17
18        Thread t2 = new Thread() -> {
19            for (int i = 0; i < 10; i++) {
20                System.out.println(Thread.currentThread().getName() + ": " + LocalDateTime.now());
21                try {
22                    Thread.sleep(1000);
23                } catch (InterruptedException e) {
24                    e.printStackTrace();
25                }
26            }
27        });
28    }
29 }
```

Run Example1

Thread-0: 20:13:16.380504
Thread-1: 20:13:16.385826
Thread-0: 20:13:17.385792
Thread-1: 20:13:17.391029
Thread-0: 20:13:18.391110
Thread-1: 20:13:18.391279
Thread-0: 20:13:19.393790
Thread-1: 20:13:19.395467
Process finished with exit code 0

Рисунок 1. Результат работы программы Example1

2) Напишите программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.



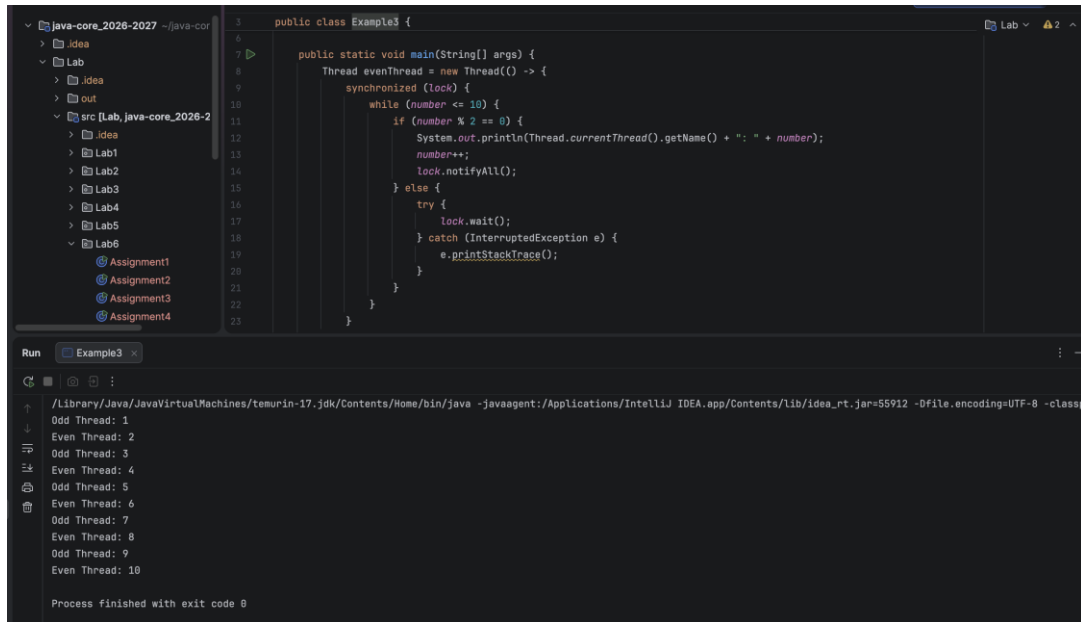
```
1 package Lab6;
2
3 public class Example2 {
4     public static void main(String[] args) throws InterruptedException {
5         Thread t = new Thread() -> {
6             for (int i = 1; i <= 10; i++) {
7                 System.out.println(i);
8                 try {
9                     Thread.sleep(1000);
10                } catch (InterruptedException e) {
11                    e.printStackTrace();
12                }
13            }
14        });
15
16        t.start();
17        t.join();
18    }
19 }
```

Run Example2

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=55901 -Dfile.encoding=UTF-8
1
2
3
4
5
6
7
8
9
10
Process finished with exit code 0

Рисунок 2. Результат работы программы Example2

3) Напишите программу, которая создает два потока, которые выводят на экран числа от 1 до 10. Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.



The screenshot displays an IDE with a project named 'java-core_2026-2027'. The source code for 'Example3' is shown in the editor. The code creates two threads: 'evenThread' and 'oddThread'. Both threads use a synchronized block with a shared 'lock' to ensure mutual exclusion. The 'evenThread' prints even numbers (2, 4, 6, 8, 10) and the 'oddThread' prints odd numbers (1, 3, 5, 7, 9). The output window shows the interleaved results of these two threads.

```
public class Example3 {  
    public static void main(String[] args) {  
        Thread evenThread = new Thread() -> {  
            synchronized (lock) {  
                while (number <= 10) {  
                    if (number % 2 == 0) {  
                        System.out.println(Thread.currentThread().getName() + ": " + number);  
                        number++;  
                        lock.notifyAll();  
                    } else {  
                        try {  
                            lock.wait();  
                        } catch (InterruptedException e) {  
                            e.printStackTrace();  
                        }  
                    }  
                }  
            }  
        }  
        Thread oddThread = new Thread() -> {  
            synchronized (lock) {  
                while (number <= 10) {  
                    if (number % 2 != 0) {  
                        System.out.println(Thread.currentThread().getName() + ": " + number);  
                        number++;  
                        lock.notifyAll();  
                    } else {  
                        try {  
                            lock.wait();  
                        } catch (InterruptedException e) {  
                            e.printStackTrace();  
                        }  
                    }  
                }  
            }  
        }  
        evenThread.start();  
        oddThread.start();  
    }  
}
```

Run Example3

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/Lib/idea_rt.jar=55912 -Dfile.encoding=UTF-8 -classpath

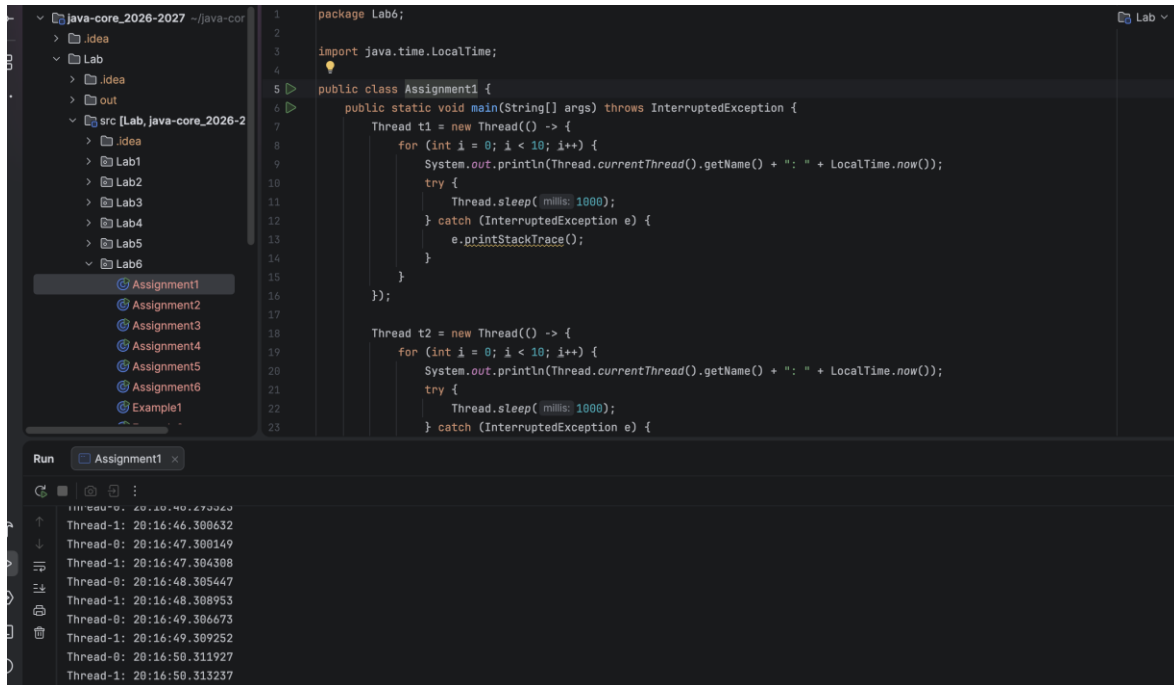
Odd Thread: 1
Even Thread: 2
Odd Thread: 3
Even Thread: 4
Odd Thread: 5
Even Thread: 6
Odd Thread: 7
Even Thread: 8
Odd Thread: 9
Even Thread: 10
Process finished with exit code 0

Рисунок 3. Результат работы программы Example3

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. Напишите программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.
2. Напишите программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.
3. Напишите программу, которая создает два потока, которые выводят на экран числа от 1 до 10. Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.
4. Напишите программу, которая создает 10 потоков и каждый поток выводит на экран свой номер.
5. Напишите функцию, которая находит максимальный элемент в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.
6. Напишите функцию, которая суммирует элементы в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.
7. Решите две любые задачи с сайта Timus.

1) Напишите программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.



The screenshot shows the IntelliJ IDEA IDE. On the left, the Project Explorer displays a directory structure with 'Lab' containing 'Assignment1' through 'Assignment6' and 'Example1'. The main editor shows the code for 'Assignment1' in the 'Lab6' package. The code creates two threads, 't1' and 't2', each running a loop from 0 to 10. Each iteration prints the thread's name and the current local time, followed by a 1000ms sleep. The 'Run' window at the bottom shows the output of the program, displaying alternating lines for 'Thread-1' and 'Thread-0' with timestamps.

```
package Lab6;

import java.time.LocalDateTime;

public class Assignment1 {

    public static void main(String[] args) throws InterruptedException {

        Thread t1 = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                System.out.println(Thread.currentThread().getName() + ": " + LocalDateTime.now());
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        };

        Thread t2 = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                System.out.println(Thread.currentThread().getName() + ": " + LocalDateTime.now());
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        };

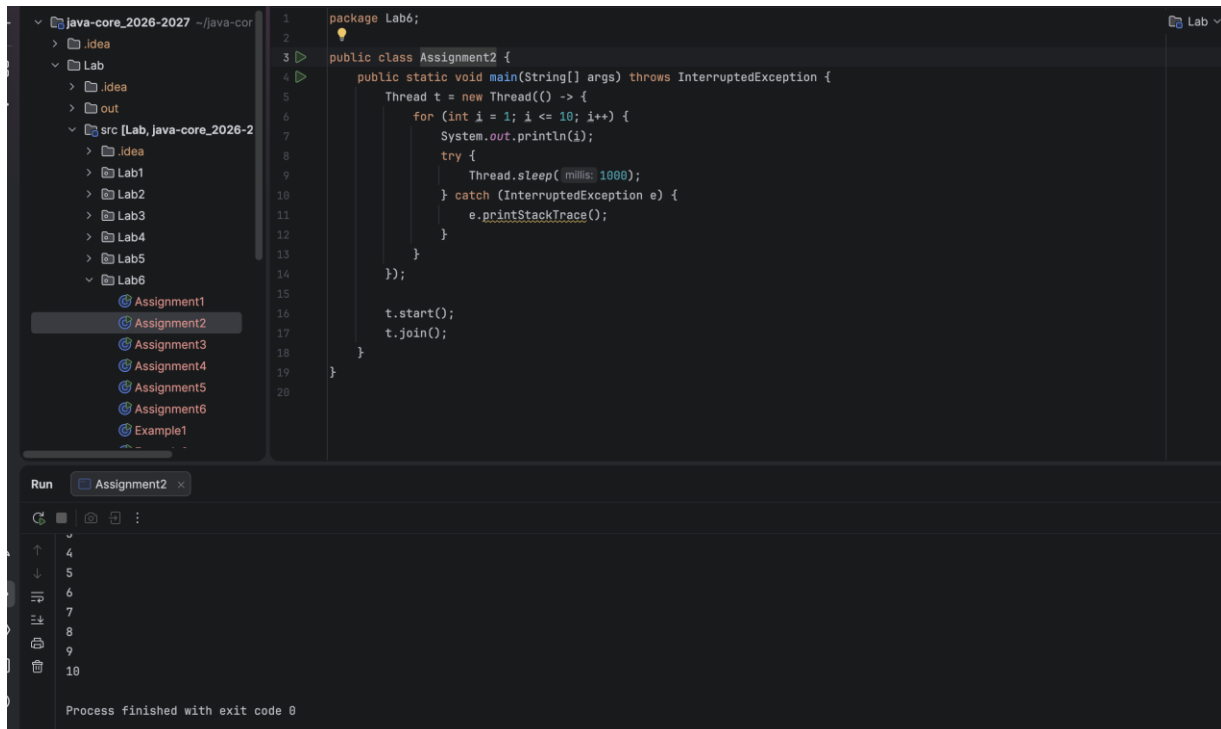
        t1.start();
        t2.start();
    }
}
```

Run Assignment1

Thread-0: 20:16:46.273363
Thread-1: 20:16:46.300632
Thread-0: 20:16:47.300149
Thread-1: 20:16:47.304308
Thread-0: 20:16:48.305447
Thread-1: 20:16:48.308953
Thread-0: 20:16:49.306673
Thread-1: 20:16:49.309252
Thread-0: 20:16:50.311927
Thread-1: 20:16:50.313237

Рисунок 4. Результат работы программы Assignment1

2) Напишите программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.



The screenshot shows the IntelliJ IDEA IDE. On the left, the Project Explorer displays a directory structure with 'Lab' containing 'Assignment1' through 'Assignment6' and 'Example1'. The main editor shows the code for 'Assignment2' in the 'Lab6' package. The code creates a single thread 't' that runs a loop from 1 to 10, printing the number 'i' and then sleeping for 1000ms. The 'Run' window at the bottom shows the output of the program, displaying numbers from 1 to 10 on separate lines.

```
package Lab6;

public class Assignment2 {

    public static void main(String[] args) throws InterruptedException {

        Thread t = new Thread() -> {
            for (int i = 1; i <= 10; i++) {
                System.out.println(i);
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        };

        t.start();
        t.join();
    }
}
```

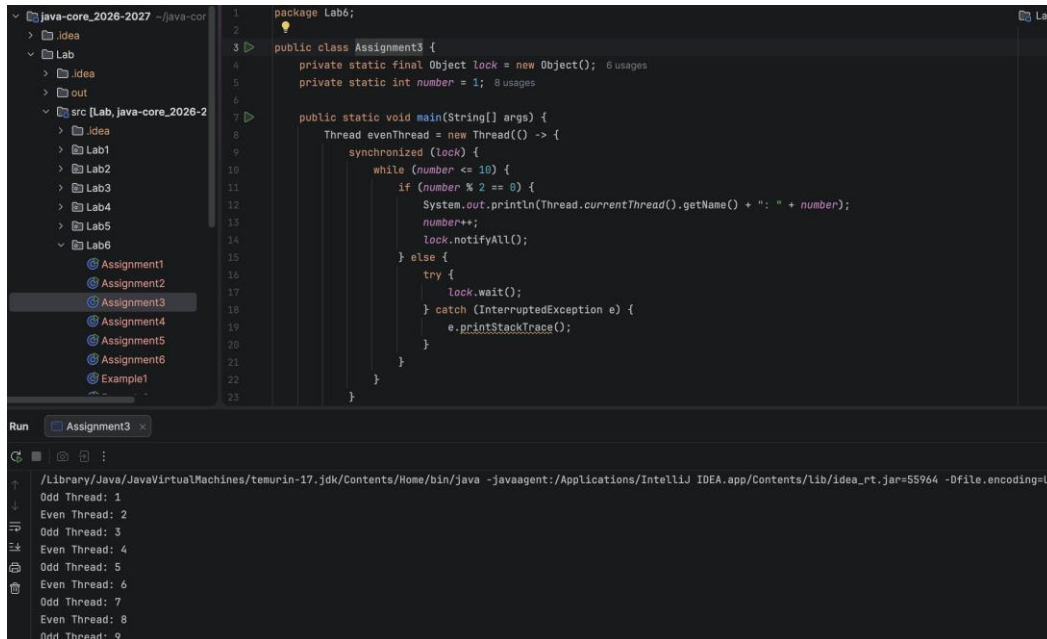
Run Assignment2

1
2
3
4
5
6
7
8
9
10

Process finished with exit code 0

Рисунок 5. Результат работы программы Assignment2

3) Напишите программу, которая создает два потока, которые выводят на экран числа от 1 до 10. Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.



The screenshot shows the IntelliJ IDEA interface. On the left, the project structure includes 'Lab6' with 'Assignment3'. The main editor displays the code for 'Assignment3.java' in the 'Lab6' package. The code creates a shared lock and a counter, then spawns two threads: 'evenThread' and 'oddThread'. 'evenThread' prints even numbers (2, 4, 6, 8, 10) and 'oddThread' prints odd numbers (1, 3, 5, 7, 9). The 'Run' window at the bottom shows the output: 'Odd Thread: 1', 'Even Thread: 2', 'Odd Thread: 3', 'Even Thread: 4', 'Odd Thread: 5', 'Even Thread: 6', 'Odd Thread: 7', 'Even Thread: 8', 'Odd Thread: 9'.

```
1 package Lab6;
2
3 public class Assignment3 {
4     private static final Object lock = new Object();
5     private static int number = 1;
6
7     public static void main(String[] args) {
8         Thread evenThread = new Thread() -> {
9             synchronized (lock) {
10                 while (number <= 10) {
11                     if (number % 2 == 0) {
12                         System.out.println(Thread.currentThread().getName() + ": " + number);
13                         number++;
14                         lock.notifyAll();
15                     } else {
16                         try {
17                             lock.wait();
18                         } catch (InterruptedException e) {
19                             e.printStackTrace();
20                         }
21                     }
22                 }
23             }
24         };
25         Thread oddThread = new Thread() -> {
26             synchronized (lock) {
27                 while (number <= 10) {
28                     if (number % 2 != 0) {
29                         System.out.println(Thread.currentThread().getName() + ": " + number);
30                         number++;
31                         lock.notifyAll();
32                     } else {
33                         try {
34                             lock.wait();
35                         } catch (InterruptedException e) {
36                             e.printStackTrace();
37                         }
38                     }
39                 }
40             }
41         };
42         evenThread.start();
43         oddThread.start();
44     }
45 }
```

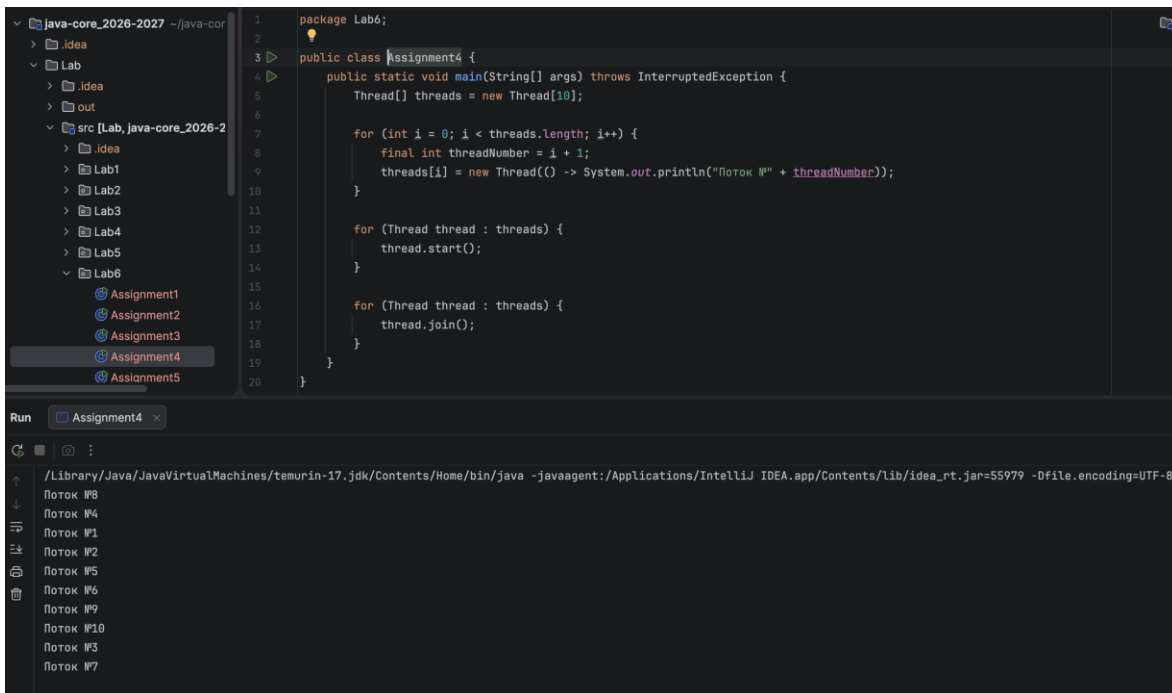
Run: Assignment3

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=55964 -Dfile.encoding=UTF-8

Odd Thread: 1
Even Thread: 2
Odd Thread: 3
Even Thread: 4
Odd Thread: 5
Even Thread: 6
Odd Thread: 7
Even Thread: 8
Odd Thread: 9

Рисунок 6. Результат работы программы Assignment3

4) Напишите программу, которая создает 10 потоков и каждый поток выводит на экран свой номер.



The screenshot shows the IntelliJ IDEA interface. On the left, the project structure includes 'Lab6' with 'Assignment4'. The main editor displays the code for 'Assignment4.java' in the 'Lab6' package. The code creates an array of 10 threads, each with a unique name ('Поток №1' to 'Поток №10'). Each thread prints its own number. The 'Run' window at the bottom shows the output: 'Поток №8', 'Поток №4', 'Поток №1', 'Поток №2', 'Поток №5', 'Поток №6', 'Поток №9', 'Поток №10', 'Поток №3', 'Поток №7'.

```
1 package Lab6;
2
3 public class Assignment4 {
4     public static void main(String[] args) throws InterruptedException {
5         Thread[] threads = new Thread[10];
6
7         for (int i = 0; i < threads.length; i++) {
8             final int threadNumber = i + 1;
9             threads[i] = new Thread() -> System.out.println("Поток №" + threadNumber));
10        }
11
12        for (Thread thread : threads) {
13            thread.start();
14        }
15
16        for (Thread thread : threads) {
17            thread.join();
18        }
19    }
20 }
```

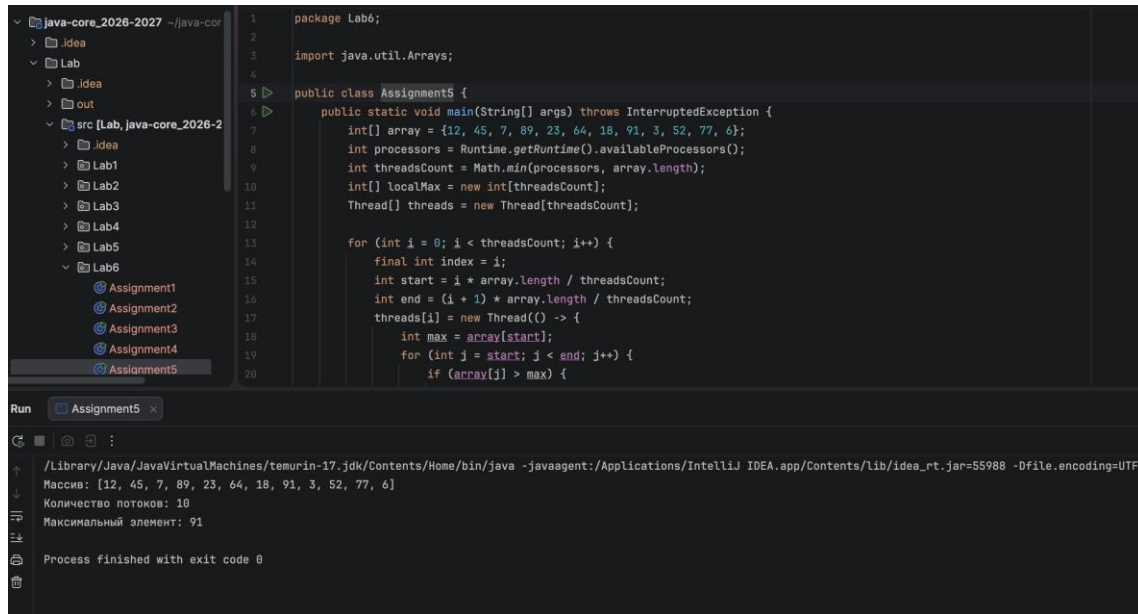
Run: Assignment4

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=55979 -Dfile.encoding=UTF-8

Поток №8
Поток №4
Поток №1
Поток №2
Поток №5
Поток №6
Поток №9
Поток №10
Поток №3
Поток №7

Рисунок 7. Результат работы программы Assignment4

5) Напишите функцию, которая находит максимальный элемент в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.



```
1 package Lab6;
2
3 import java.util.Arrays;
4
5 public class Assignment5 {
6     public static void main(String[] args) throws InterruptedException {
7         int[] array = {12, 45, 7, 89, 23, 64, 18, 91, 3, 52, 77, 6};
8         int processors = Runtime.getRuntime().availableProcessors();
9         int threadsCount = Math.min(processors, array.length);
10        int[] localMax = new int[threadsCount];
11        Thread[] threads = new Thread[threadsCount];
12
13        for (int i = 0; i < threadsCount; i++) {
14            final int index = i;
15            int start = i * array.length / threadsCount;
16            int end = (i + 1) * array.length / threadsCount;
17            threads[i] = new Thread(() -> {
18                int max = array[start];
19                for (int j = start; j < end; j++) {
20                    if (array[j] > max) {
```

Run Assignment5 x

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=55988 -Dfile.encoding=UTF-8

Массив: [12, 45, 7, 89, 23, 64, 18, 91, 3, 52, 77, 6]

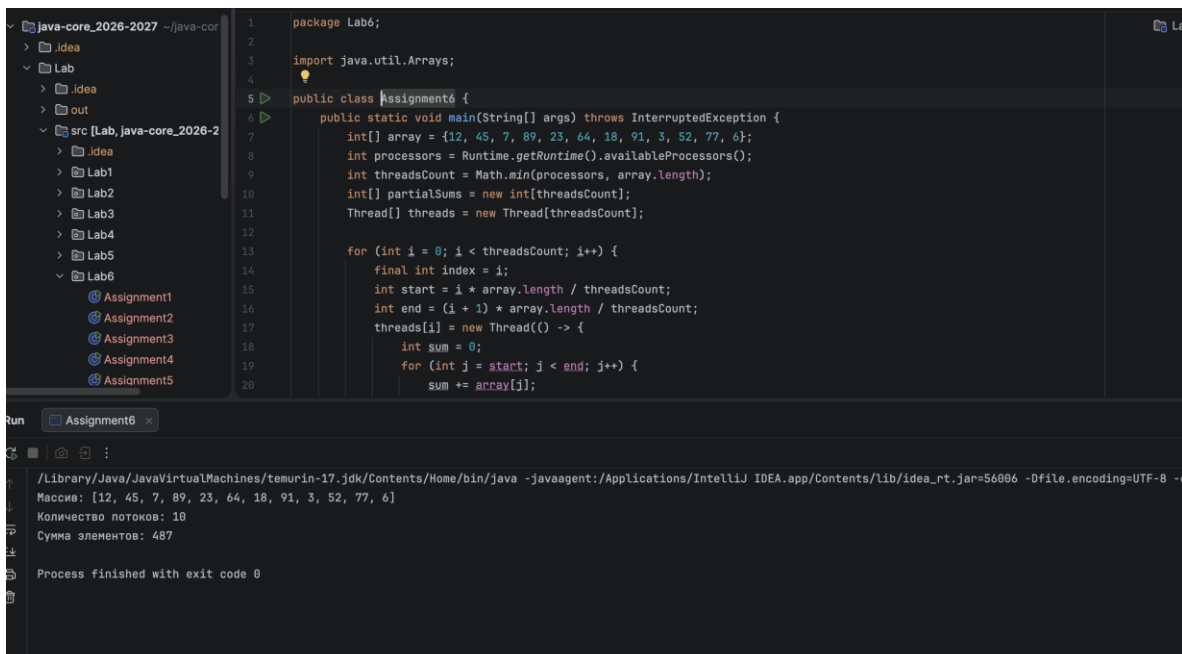
Количество потоков: 10

Максимальный элемент: 91

Process finished with exit code 0

Рисунок 8. Результат работы программы Assignment5

6) Напишите функцию, которая суммирует элементы в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.



```
1 package Lab6;
2
3 import java.util.Arrays;
4
5 public class Assignment6 {
6     public static void main(String[] args) throws InterruptedException {
7         int[] array = {12, 45, 7, 89, 23, 64, 18, 91, 3, 52, 77, 6};
8         int processors = Runtime.getRuntime().availableProcessors();
9         int threadsCount = Math.min(processors, array.length);
10        int[] partialSums = new int[threadsCount];
11        Thread[] threads = new Thread[threadsCount];
12
13        for (int i = 0; i < threadsCount; i++) {
14            final int index = i;
15            int start = i * array.length / threadsCount;
16            int end = (i + 1) * array.length / threadsCount;
17            threads[i] = new Thread(() -> {
18                int sum = 0;
19                for (int j = start; j < end; j++) {
20                    sum += array[j];
```

Run Assignment6 x

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=56806 -Dfile.encoding=UTF-8 -c

Массив: [12, 45, 7, 89, 23, 64, 18, 91, 3, 52, 77, 6]

Количество потоков: 10

Сумма элементов: 487

Process finished with exit code 0

Рисунок 9. Результат работы программы Assignment6

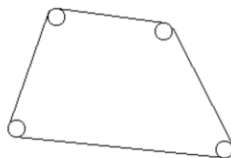
РЕШЕНИЕ ЗАДАЧ TIMUS

1020. Ниточка

Ограничение времени: 1.0 секунды

Ограничение памяти: 64 МБ

Злоумышленники варварски вбили в ни в чем не повинную плоскую поверхность N гвоздей, да так, что только шляпки остались. Мало того, они в своих подлых целях вбили все гвозди в вершины выпуклого многоугольника. После этого они... страшно сказать... они натянули ниточку вокруг всех гвоздей, так, что поверхности стало совсем больно! Вот как примерно они это сделали:



Ваша задача — определить длину этой ниточки.

Исходные данные

В первой строке входа к этой задаче находятся два числа — количество гвоздей N , $1 \leq N \leq 16$, и вещественное число R — радиус шляпок гвоздей. Все шляпки имеют одинаковый радиус. Далее на входе располагаются еще N строк, в каждой из которых записана через пробел пара вещественных координат центра очередного гвоздя; координаты не превосходят по абсолютной величине числа 100. Описания гвоздей приводятся в порядке обхода вершин многоугольника (либо по часовой стрелке, либо против часовой стрелки), начиная с произвольного. Шляпки разных гвоздей не накладываются друг на друга.

Результат

Выведите вещественное число, округлённое до двух знаков после запятой — длину ниточки, натянутой вокруг всех гвоздей.

Пример

исходные данные	результат
4 1 0.0 0.0 2.0 0.0 2.0 2.0 0.0 2.0	14.28

Рисунок 10. Условие задачи Timus 1020

```
1 package timus;
2
3 import java.util.Scanner;
4
5 public class Timus1020 {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8
9         int n = scanner.nextInt();
10        double r = scanner.nextDouble();
11
12        double[] x = new double[n];
13        double[] y = new double[n];
14
15        for (int i = 0; i < n; i++) {
16            x[i] = scanner.nextDouble();
17            y[i] = scanner.nextDouble();
18        }
19
20        double length = 0;
```

Timus1020 x

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=56210 -Dfile.encoding=UTF-8
4 1
0.0 0.0
2.0 0.0
2.0 2.0
0.0 2.0
14.28

Process finished with exit code 0
```

Рисунок 11. Решение задачи Timus 1020

Вступление

— Брат мой, Магистр Ордена хочет узнать завтра о результатах наших многолетних изысканий. Он хочет видеть, ни много, ни мало, Суммирующую Машину! Даже более того: он хочет, чтобы наша Машина — всего лишь машина — продемонстрировала свое постижение Таинства Суммы настолько глубоко, насколько это возможно. Он хочет, чтобы Машина нашла каких-нибудь два числа, дающих в сумме священное число 10000!

— Тс-с-с! Но это же безумство, граничащее с кощунством! Как Машина может ВЫЧИСЛИТЬ священное число? Двадцать семь лет мы работаем над ней, и смогли только лишь научить ее отвечать на вопрос: «Больше сумма двух введенных чисел, чем 10000, или меньше?». Но разве может смертный найти два таких числа, чтобы их сумма оказалась равна 10000?

— И все же нам придется сделать это с помощью нашей Машины, пусть она и неспособна на это. Иначе у нас будут... ну, скажем так, крупные неприятности, если кипящее масло можно назвать таким словом. Впрочем, у меня есть идея. Помнишь, на той неделе мы ввели в Машину числа -7 и 13 , она ответила, что их Сумма меньше 10000. Я не знаю, как это проверить, но нам ничего не остается, как доверять созданию наших рук. Что, если теперь мы возьмем число большее, чем -7 и снова запустим Машину? И будем так делать снова и снова, пока не найдем такое число, которое в сумме с 13 даст 10000! Надо только подготовить список возрастающих чисел.

— Не верю я в эту идею... Давай лучше начнем с суммы, заведомо большей, чем Священное число и будем уменьшать одно из слагаемых, так у нас больше шансов избежать кипяще... крупных неприятностей.

Так ни о чём и не договорившись, Братья разошлись по своим кельям. К следующему дню, каждый из них подготовил такой список чисел, который, по его мнению, мог бы их спасти... Смогут ли спасти их оба списка вместе?

Задача

Ваша программа должна определять, можно ли из двух списков целых чисел выбрать по одному числу так, чтобы в сумме они составили 10000.

Исходные данные

Состоят из двух списков — одного, потом другого. Формат каждого из этих списков таков: в первой строчке записано количество N_i чисел в i -м списке, далее в N_i строчках по одному числу в строке записаны сами списки. Выполняются неравенства $1 \leq N_i \leq 50000$, все элементы списков лежат в диапазоне от -32768 до 32767 . Первый список упорядочен по возрастанию, второй — по убыванию.

Результат

На выходе следует записать YES, если из списков можно выбрать по числу, которые в сумме дадут 10000 и NO в противном случае.

Пример

исходные данные	результат
4 -175 19 19 10424 3 8951 -424 -788	YES

Рисунок 12. Условие задачи Timus 1021

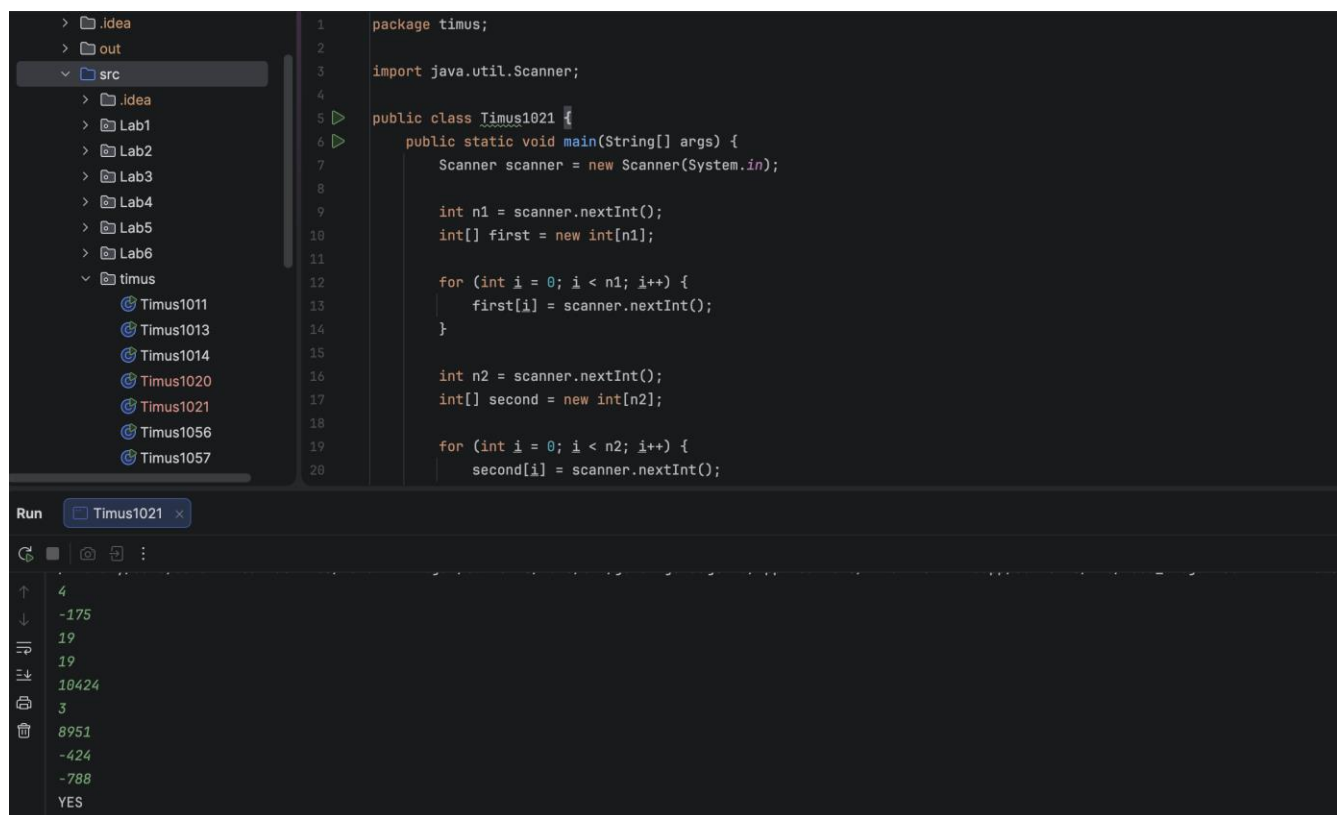


Рисунок 13. Решение задачи Timus 1021

ВЫВОД

В ходе выполнения лабораторной работы был изучен механизм многопоточности в языке программирования Java. Были рассмотрены способы создания и запуска потоков, применение методов `sleep()`, `start()` и `join()`, а также организация синхронизации потоков с помощью `wait()` и `notifyAll()`. В ходе самостоятельной работы были реализованы программы для запуска нескольких потоков, вывода чисел с задержкой, разделения работы между потоками, поиска максимального элемента массива и суммирования элементов массива с использованием количества потоков, равного количеству ядер процессора.